



AutoBench: Automatic Testbench Generation and Evaluation Using LLMs for HDL Design

Ruidi Qiu
Technical University of Munich
r.qiu@tum.de

Grace Li Zhang
TU Darmstadt
grace.zhang@tu-darmstadt.de

Rolf Drechsler
University of Bremen
drechsler@uni-bremen.de

Ulf Schlichtmann
Technical University of Munich
ulf.schlichtmann@tum.de

Bing Li
University of Siegen
bing.li@uni-siegen.de

Abstract

In digital circuit design, testbenches (TBs) constitute the cornerstone of simulation-based hardware verification. Traditional methodologies for testbench generation during simulation-based hardware verification still remain partially manual, resulting in inefficiencies in testing various scenarios and requiring expensive time from designers. Large Language Models (LLMs) have demonstrated their potential in automating the circuit design flow. However, directly applying LLMs to generate testbenches suffers from a low pass rate. To address this challenge, we introduce AutoBench, the first LLM-based testbench generator for digital circuit design, which requires only the description of the design under test (DUT) to automatically generate comprehensive testbenches. In AutoBench, a hybrid testbench structure and a self-checking system are realized using LLMs. To validate the generated testbenches, we also introduce an automated testbench evaluation framework to evaluate the quality of generated testbenches from multiple perspectives. Experimental results demonstrate that AutoBench achieves a 57% improvement in the testbench pass@1 ratio compared with the baseline that directly generates testbenches using LLMs. For 75 sequential circuits, AutoBench successfully has a 3.36 times testbench pass@1 ratio compared with the baseline. The source codes and experimental results are open-sourced at this link: <https://github.com/AutoBench/AutoBench>. Artifact DOI: 10.5281/zenodo.13325723.

Keywords

Large Language Model, Hardware Simulation, Testbench Generation

ACM Reference Format:

Ruidi Qiu, Grace Li Zhang, Rolf Drechsler, Ulf Schlichtmann, and Bing Li. 2024. AutoBench: Automatic Testbench Generation and Evaluation Using LLMs for HDL Design. In *2024 ACM/IEEE International Symposium on Machine Learning for CAD (MLCAD '24)*, September 9–11, 2024, Salt Lake City, UT, USA. ACM, New York, NY, USA, 10 pages. <https://doi.org/10.1145/3670474.3685956>



This work is licensed under a Creative Commons Attribution International 4.0 License. *MLCAD '24, September 9–11, 2024, Salt Lake City, UT, USA*
© 2024 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-0699-8/24/09
<https://doi.org/10.1145/3670474.3685956>

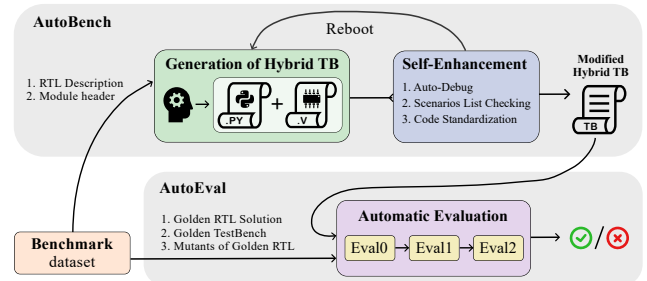


Figure 1: Outline of AutoBench workflow and AutoEval evaluation framework.

1 Introduction

Simulation-based verification is one of the most common techniques for hardware functional verification [23]. This verification is carried out using testbenches to validate the functionality of a DUT. According to recent studies by the Wilson Research Group [7], around 49% of a design engineer's time in an IC/ASIC project is spent on conducting verification tasks. Thus, automating the generation of testbenches is a key point of automating the whole EDA design process of digital circuits. Previous efforts, such as [12, 13], have sought to automate the code-writing process. But they still need the test stimulus and reference signals designed by engineers. Random test case generation approaches, such as constrained random generation (CRG) [10], have also been proposed to reduce the human effort on test stimulus generation, but the reference signals for checking are still needed from humans. Thus, these studies could not fully automate testbench generation, including both the selection of test vectors and the checking of DUT's signals.

Due to the limitations of conventional algorithms, previous research has been unable to achieve complete automation of testbench generation. With the growing trend of AI applications in hardware design [5], particularly the advancements facilitated by LLMs, recent studies [2, 3, 11, 18] on RTL generation using LLMs highlight the proficiency and knowledge of LLMs in digital hardware design. Additionally, several studies have explored the application of LLMs in the verification process. For instance, [15, 16] investigate the potential of LLMs in formal verification, while [25] employs LLMs to generate hardware test stimuli. [22] offers a case study on LLM-based processor verification. Despite these efforts, a systematic approach for LLM-automated testbench generation remains absent, leaving a blank in the automation of the entire simulation-based verification process.

In this paper, an LLM-based testbench generation workflow, *AutoBench*, is proposed to automatically generate testbenches for

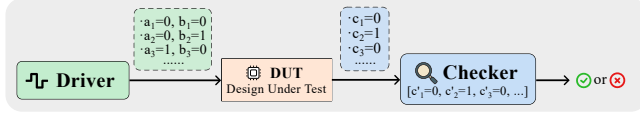


Figure 2: The driver and the checker in a TB, assuming the DUT has two input ports “a” and “b” and one output port “c”.

simulation-based verification. A testbench evaluation workflow, *AutoEval*, is introduced to automatically assess the generated testbenches using multiple metrics. *AutoEval* can also be used to test the quality of testbenches generated by other testbench generation frameworks. The outline of *AutoBench* and *AutoEval* is illustrated in Fig. 1. The contributions of this paper are summarized as follows:

- This is the first **systematic and generic** work to generate Verilog testbenches using LLMs for RTL verification. Unlike previous work on LLM-based verification, our work is validated on a large and widely used dataset.
- A **hybrid testbench architecture**, including LLM-generated Python code, LLM-generated Verilog code, and Python script, is proposed in our work.
- A comprehensive LLM-based **code generation method** is introduced in our work, including hybrid code generation, scenario checking, code standardization, and LLM-based automatic code debugging and rebooting.
- An automatic evaluation framework, *AutoEval*, including a series of general LLM-generated testbench **evaluating metrics**, is proposed in our work. In addition, a **dataset** for *AutoEval* is proposed, which is extended with the help of LLMs from the dataset in previous work [11].
- The code implementation, the dataset and the experimental results in this work are all **open-sourced** at <https://github.com/AutoBench/AutoBench>.

The rest of the paper is organized as follows. In Section 2, the background of this work is explained. In Section 3, the proposed *AutoBench* workflow to generate testbenches is explained. In Section 4, the proposed *AutoEval* framework to evaluate the testbench generation framework is elaborated. Experimental results and conclusions are shown in Section 5 and Section 6. The detailed demonstrations of *AutoBench* forward generation process are shown in the Appendix.

2 Background and Motivation

2.1 Testbench Generation

The conventional testbench design can be split into two parts: the *driver* design and the *checker* design, as shown in Fig. 2. The function of the driver is to generate the input signal vectors (or, in other words, stimuli), drive DUT to generate output signal vectors, and export these vectors to the checker at a proper time. The checker is responsible for checking if the DUT behaves as required, i.e., checking the output signals from DUT.

An appropriate testbench should be exhaustive and accurate [6]. *Exhaustive* implies that the testbench is capable of covering a sufficient number of testing scenarios, while *accurate* denotes that under a given test stimulus, the testbench can provide a correct assessment.

There are several approaches to measure how exhaustive a testbench is, including code coverage, circuit coverage and mutant

coverage [6]. The coverage metrics can also be used in test generation [9, 24].

2.2 Challenges in LLM-based TB Generation

LLMs still encounter challenges such as laziness [17], hallucination [8] and insufficient training data on hardware design, making the direct generation of testbenches for complex DUTs ineffective. For instance, due to LLM laziness, the generated testbench may end prematurely after listing only a few test cases, resulting in low coverage. Additionally, the reference signals required to verify the DUT are not always clear from DUT’s specification. In complex cases, context-based LLMs may guess incorrect values due to hallucination. Furthermore, commercial LLMs such as GPT-4 Turbo are trained more extensively on software codes than hardware codes, indicating a weak performance in hardware contexts. To address issues of laziness and hallucination, *AutoBench* divides the testbench generation process into multiple stages, applying the idea of chain-of-thought [19]. Additionally, our hybrid testbench structure uses software code, leveraging the advantage of more comprehensive training data in the software domain for a better performance.

3 Automatic Testbench Generation Framework

In this section, the workflow of *AutoBench* is explained, including two parts: testbench forward generation in Section 3.1 and testbench self-enhancement in Section 3.2, as is depicted in Fig. 3. The information we have as input to the *AutoBench* workflow includes the DUT’s problem description (or, in other words, RTL description) and the module header, as shown in Fig. 8, where the circuit type (combinational or sequential) is generated by the *AutoBench* workflow. In generating a testbench, the DUT is not available to *AutoBench*. Otherwise, errors in the DUT may misguide the LLMs so that the resulting testbench may ignore errors. The following explanation of *AutoBench* workflow is accompanied by an example of the testbench generation task *gates100* originally from HDLBits [21]. The detailed demonstrations of this testbench generation task include prompts and LLM’s responses in each stage, as shown in the Appendix.

3.1 Forward Generation Workflow

The fundamental concept of *AutoBench* is to emulate the design process used by human engineers. As outlined in Fig. 2, the primary generation pipeline is split into two distinct tracks: the driver track and the checker track. The major steps of the testbench generation workflow in Fig. 3 are explained in detail below.

3.1.1 Circuit Type Discriminator Digital logical circuits are classified into two types: combinational (CMB) circuits and sequential (SEQ) circuits. The circuit type can be derived from the DUT’s description, even without DUT’s code. Identifying the circuit type initially allows *AutoBench* to provide more precise guidance in subsequent stages to enhance the accuracy of LLM-generated testbenches. For instance, testbenches for sequential circuits require stringent time-dependent functions, while a checker for a combinational circuit can be a direct function of the current input of the DUT.

To discriminate the types of the DUT, *AutoBench* generates its own code sample and uses it to decide whether the target DUT is a sequential design or a combinational design. As shown at the top left corner of Fig. 3, given the Benchmark Suite, including

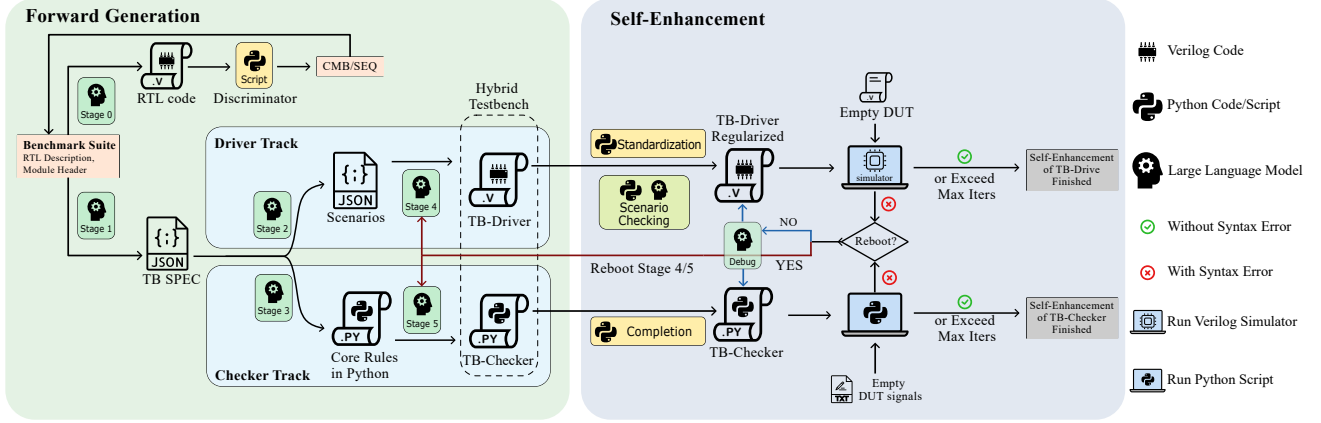


Figure 3: AutoBench: The TB generation workflow in detail.

the problem description and module header, the LLM is guided to directly generate the RTL code for DUT at **stage 0**. Because the generated RTL is only used for the discriminator in the next step, the correctness of the generated code does not have to be very high as long as there are sufficient features for the later discrimination. As the Verilog syntax is strictly restricted by IEEE standard [1], the circuit type of an RTL code can be simply determined by special code statements using a Python script. For example, consider a simple instance of `always@(posedge signalx)`, where “signalx” represents any signal. This is always a sequential statement. The determined circuit type either CMB or SEQ is added to the *Benchmark Suite* to influence the detailed prompts in the following stages, as shown in Fig. 8.

3.1.2 Verilog Driver Track The driver of AutoBench is a Verilog file, similar to a part of a conventional Verilog testbench file. The difference is that the driver only inputs the test stimuli into the DUT and exports its output. No correctness check on the DUT’s output happens in this stage, which is left to the checker part of AutoBench. The technical requirements (TRs) for an AutoBench driver are as follows: **TR1**, the driver’s test stimuli must achieve a high coverage rate; **TR2**, the driver should drive the DUT and collect its signals at appropriate times, which is especially important in sequential circuits; **TR3**, the driver should export the DUT’s signals in a specified format to be processed by the checker.

After the type of the DUT is identified, the *Benchmark Suite* contains the RTL description and the module header of the DUT, along with the circuit type. This information pertains specifically to the RTL code of the DUT rather than the testbench directly, and it lacks certain explicit details, such as a description of the DUT from the perspective of a testbench designer and strategies for designing testbench. Without these specific details, the LLM may generate an ineffective testbench, ultimately leading to a verification failure. To enable the LLM’s comprehension of the testbench generation task, we instruct the LLM to summarize a testbench specification in the JSON format from the Benchmark Suite in the next step **stage 1**, as shown in Fig. 3. An example of testbench specification generation is shown in Fig. 9.

Although the design strategies are already contained in the testbench specification, designing the driver simply according to the abstract strategies may cause the laziness of LLMs in choosing test stimuli. To design test stimuli in the driver with a high coverage

to fulfill TR1, a list describing detailed test scenarios is needed to reference the design to the driver code. Thus, in **stage 2**, the LLM is directed to generate a list of test scenarios from the information in testbench specification. A demonstration of stage 2 is shown in Fig. 10(a). Compared to directly generating the testbench, this approach allows the LLM to focus on coverage with split scenarios, thereby achieving a higher coverage ratio.

Subsequently, in **stage 4** (stage 3 is in Python checker track, which will be introduced later), the LLM ultimately produces the Verilog driver that drives the DUT and collects signals at the correct time points, as is required by TR2, utilizing the testbench specification, test scenarios, and information from the Benchmark Suite. An example of generating a Verilog driver in stage 4 is shown in Fig. 10(b). The instruction for exporting the DUT’s signals into a .txt file is also provided in the prompt, thus satisfying TR3. These signals in the .txt file will be read by the Python checker later to compare the outputs of the DUT and the expected values.

For sequential DUTs, to verify their correctness under a certain test scenario, the testbench needs to access the DUT’s signals from not only the current time point but also the previous time points. Wrong or missing time points to export the signals may mislead the Python checker later and make the checking process fail. Thus, the generation of Verilog drivers for sequential circuits is more complex than for combinational circuits. Subsequently, Stage 4 for sequential circuits is divided into two steps. The target of the first step is generating the architecture of the driver code and adding the `$fdisplay` statements to export the DUT’s signals at the checking time points. The second step is to assert the `$fdisplay` statements to export signals at the previous time points before checking. An example of stage 4 for sequential DUTs is demonstrated in Fig. 12 and Fig. 13 in the Appendix.

3.1.3 Python Checker Track With the DUT’s signals driven by test stimuli from the Verilog driver, the next step in AutoBench is to generate the expected output signals with respect to the stimuli based on the DUT’s description, while the DUT code is still not involved in this track.

Since the checker aims to verify whether the outputs of the DUT with respect to the stimuli are correct, the checker in AutoBench only needs to generate the expected output signals and compare them with the actual output of the DUT when the testbench is applied. Accordingly, it is not necessary to express the checker

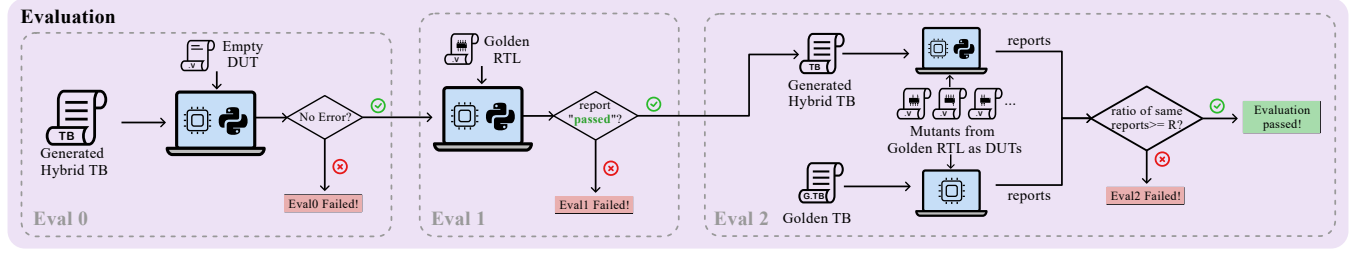


Figure 4: AutoEval: The evaluation framework in detail.

in hardware description languages. Instead, we use Python as the language for the checker.

The utilization of Python code as a testbench checker offers three technical advantages (TAs). **TA1 - more appropriate**, because Python, being a higher-level language, provides a greater level of abstraction while ignoring circuit details, making it more suitable for verification tasks than Verilog. **TA2 - easier**, since Python benefits from a more extensive dataset than Verilog when LLMs are trained, resulting in a generally superior performance of general conversational language models in Python over Verilog. In addition, Python’s extensive standard and third-party libraries make it inherently easier to code than Verilog. **TA3 - orthogonal**, because the programming paradigm of Python differs significantly from that of Verilog, which can effectively prevent potential conflicts and errors that may arise from using Verilog codes to verify other Verilog codes. For instance, the designers may make the same error as the DUT’s Verilog code when writing the reference RTL’s Verilog code for testbench.

As illustrated in the bottom left of Fig. 3, the complexity of the Python checker generation task is mitigated by dividing the Python code generation process into two distinct stages. The former stage, referred to as **stage 3**, guides LLM to *translate* the core checking rules of the testbench in natural language into Python, utilizing the testbench specification from stage 1 and other information from Benchmark Suite. The latter stage, termed **stage 5**, involves the generation of the complete Python checker code by leveraging the code produced in stage 3, along with the same information used in stage 3, as the prompt to the LLM. Examples of stage 3 and stage 5 are shown in Fig. 11(a) and Fig. 11(b), respectively.

3.2 Self-Enhancement after generation

With the generated hybrid testbench, implementing a self-enhancement system is necessary to enhance the testbench. Note that the self-enhancement system does not involve functional correction for the Python checker because there is no additional data to verify the correctness of the testbench in practice.

3.2.1 Code Completion and Standardization The Python code from stage 5 is completed by adding a fixed signal interface function, which can read the signals in the .txt file generated by the Verilog driver from stage 4 and send them to the checking function in the Python checker. For sequential circuits, the Verilog driver must be standardized due to the complexity of the required format, which language models cannot perfectly implement. For instance, stage 4’s prompt requires the driver to export sequential DUT inputs every clock cycle to ensure that the checker has sufficient information, which is often neglected by LLMs. The standardization script addresses this by dividing long delays and inserting “\$display” statements.

3.2.2 Scenario Checking During the integration of test scenarios into the driver code in stage 4, there is a probability that the LLM may omit some scenarios and take shortcuts due to laziness mentioned in Section 2.2, thus significantly reducing the testbench’s coverage rate. To avoid such negligence, scenario checking has been incorporated into our workflow, as is shown in the middle of Fig. 3. Scenario checking uses a Python script to check if all scenarios are included in the Verilog driver code. If incomplete, the scenario list from stage 2 and the partial driver code are provided to the LLM to complete the code. This process iterates up to a maximum of I_{SC} iterations, set to 3 in Section 5.

3.2.3 Auto Debugging and Rebooting Although the functional correctness of the generated testbench cannot be verified, its syntactic correctness can be confirmed using a Verilog simulator and a Python interpreter. As illustrated in the upper right of Fig. 3, the driver code is initially simulated with an empty DUT code, which includes only the module header but no content. If a syntax error in the testbench arises during this process, the error message and the Verilog code with line markers are provided to the LLM to attempt a resolution using its Verilog knowledge. However, the LLM is not capable of rectifying all syntax errors. Consequently, a rebooting system is activated after every I_R debugging attempts. Upon activation, the system reverts to stage 4 to regenerate the Verilog driver. The debugging process for the Python checker is analogous to that of the driver, but the rebooting returns to stage 5 to regenerate the final checker code, as depicted at the bottom middle of Fig. 3. Additionally, in some cases, the failure of running the checker code is attributable to the driver code, such as an incorrect signal format transferred to the checker. To address this issue, I_{RV} instances of Python errors will trigger the rebooting of the Verilog driver. The iterations of debugging and rebooting in total are capped at I_D for each type of code. In this study, I_R , I_{RV} , and I_D are set to 1, 2, and 5, respectively.

4 Evaluation Framework for Testbenches

A testbench can assess RTL code correctness, but no *hyper-testbench* exists to evaluate the testbench’s correctness. This study introduces *AutoEval*, a framework with three criteria to assess testbenches under evaluation (TUEs) generated by LLMs. AutoEval is versatile, evaluating both hybrid testbenches from the AutoBench framework and conventional Verilog testbenches. The evaluation framework is shown in Fig. 4.

The first criterion, **Eval0**, ensures the TUE is syntactically correct and can be compiled. The second criterion, **Eval1**, evaluates the preliminary correctness of the TUE. It reports a status of *passed* if the TUE detects no errors when the golden RTL code is utilized as the DUT.

Table 1: Main results of proposed AutoBench framework.

Group	Metric	Pass@1				Pass@5				Pass@10			
		Ratio (%)		#Tasks		Ratio (%)		#Tasks		Ratio (%)		#Tasks	
		Ours	Baseline	Ours	Baseline	Ours	Baseline	Ours	Baseline	Ours	Baseline	Ours	Baseline
Total (156)	Eval2	44.81%	28.46%	69.9	44.4	69.38%	55.78%	108.2	87.0	76.92%	65.38%	120.0	102.0
	Eval1	51.47%	41.73%	80.3	65.1	81.25%	81.48%	126.8	127.1	88.46%	92.95%	138.0	145.0
	Eval0	95.71%	70.06%	149.3	109.3	99.97%	98.22%	156.0	153.2	100.00%	100.00%	156.0	156.0
CMB (81)	Eval2	62.22%	47.65%	50.4	38.6	83.39%	80.82%	67.5	65.5	87.65%	86.42%	71.0	70.0
	Eval1	64.81%	58.52%	52.5	47.4	87.39%	93.44%	70.8	75.7	93.83%	97.53%	76.0	79.0
	Eval0	94.20%	83.58%	76.3	67.7	99.94%	99.83%	81.0	80.9	100.00%	100.00%	81.0	81.0
SEQ (75)	Eval2	26.00%	7.73%	19.5	5.8	54.25%	28.74%	40.7	21.6	65.33%	42.67%	49.0	32.0
	Eval1	37.07%	23.60%	27.8	17.7	74.62%	68.58%	56.0	51.4	82.67%	88.00%	62.0	66.0
	Eval0	97.33%	55.47%	73.0	41.6	100.00%	96.49%	75.0	72.4	100.00%	100.00%	75.0	75.0

Table 2: Definitions of Proposed Evaluation Criterion

Type	Definition
Failed	codes have syntax error
Eval0	codes have no syntax error
Eval1	codes passed Eval0; report <i>passed</i> with the golden RTL code as DUT
Eval2	codes passed Eval1; use mutants of golden RTL as DUTs; have the same report as the golden testbench (<i>passed</i> or <i>failed</i>)
Eval2b	similar to Eval2 but use RTL codes generated by LLM as DUTs

However, the aforementioned criteria only identify certain incorrect TUEs and do not address incomplete coverage. For example, a TUE that simply reports *no error* for any DUT but without checking the actual signals would always pass both Eval0 and Eval1. Thus, an additional coverage-focused criterion is necessary. Additionally, to ensure objectivity and efficiency, this criterion should be automated. Inspired by the mutant coverage metric [6], we propose **Eval2**, a mutant-based coverage-oriented testbench evaluation criterion. In Eval2, the DUTs are mutants of the golden RTL code generated by the LLM. Both the TUE and the golden testbench (GTB) are run concurrently for each mutant DUT. Their results (*passed* or *failed*) are compared. The TUE is marked as *success* for that DUT if the results of the TUE and GTB match. The proportion of *successes* is the ratio of matches between the TUE and GTB with respect to all the DUT mutants, which represents the coverage of the TUE. A testbench passes Eval2 if the Eval2 ratio exceeds a threshold R , set to 80% in Section 5.

The mutants of the golden RTL codes are generated by the LLM. The LLM is provided with the golden RTL code and asked to generate N_m mutants by making minor modifications evenly on the code. To provide the LLM with a better understanding of the code, the RTL description is also provided as a prompt. In this work, the mutant number N_m is set to 10. For some very simple codes, the finally generated mutants could be less than N_m . The definitions of evaluation criteria are summarized in Table 2, where the criterion Eval2b will be discussed in Section 5.4.

5 Experimental Results

5.1 Experimental Setup

5.1.1 Software Environment and LLM Model Selection In this work, ICARUS Verilog [20] was used as the Verilog simulator, which supports IEEE1800-2012 standards, including system verilog. All the

scripts and Python checkers were executed on *Python 3.8.10 64-bit*. All the experiments were conducted on *gpt-4-turbo-2024-04-09* [14].

5.1.2 Evaluation Metrics Our work is evaluated under multiple evaluation criteria from AutoEval, including *Eval0*, *Eval1* and *Eval2*, as described in Section 4 and Table 2. The *pass@k* metric is used, where a testbench generation task is considered to pass an evaluation criterion if k testbenches for the same task are generated and at least one testbench passes this criterion. To reduce the variance from LLM, the unbiased estimator from [4] is used:

$$pass@k = \mathbb{E}_{Problems} \left[1 - \frac{\binom{n-c}{k}}{\binom{n}{k}} \right] \quad (1)$$

where n is the total samples we run for each problem and should be as large as possible to guarantee the quality of the testbenches. In this work, n was set to 10, considering the cost of using the API of LLMs.

5.1.3 Dataset The dataset used in this study is an extended version of VerilogEval-Human [11], an RTL dataset comprising 156 Verilog problems derived from HDLBits [21]. This extension incorporates RTL mutant codes to facilitate Eval2, as discussed in Section 4.

5.2 Main Results

5.2.1 Introduction of Baseline To demonstrate the performance of the proposed AutoBench, a comparison experiment between AutoBench workflow and a baseline workflow was conducted. The baseline involves a one-step workflow where an LLM receives the *code format*, *problem description*, along with the *DUT header*, and generates the testbench directly.

5.2.2 Main Results The results of the comparison experiment are shown in Table 1. The first column *group* denotes the circuit type. The Second column *metric* represents the different evaluation criteria introduced in AutoEval. *Ratio* and *#Tasks* refer to the pass percentage and pass number of test cases, respectively. In terms of Eval2 pass@1 ratio, which is the most important metric in practice, the third and fourth columns in the fourth row indicate that AutoBench achieves a 57% ($\frac{44.81\%}{28.46\%} - 1$) improvement compared with the baseline. In addition, for sequential tasks, AutoBench has a 3.36 ($26.00\% \div 7.73\%$) times Eval2 pass@1 ratio compared with the baseline (columns 3/4 in row 10). Due to the higher complexity and the need to consider timing order, both AutoBench and the baseline have lower performance for sequential circuits than for

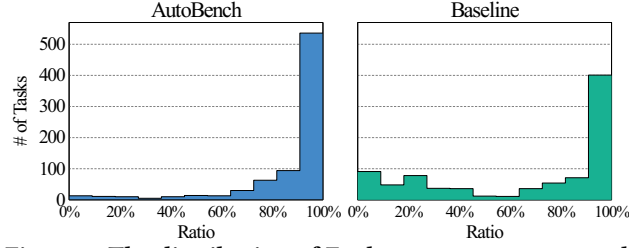


Figure 5: The distribution of Eval2 coverages among tasks passing Eval1

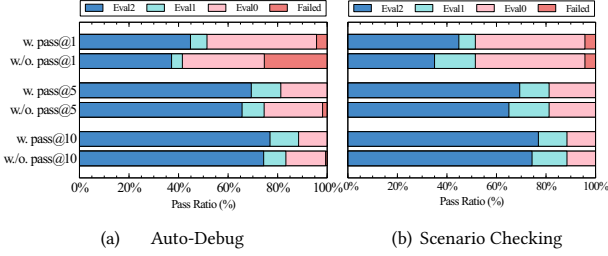


Figure 6: Ablation Studies of self-enhancement methods: (a) Auto-Debug and (b) Scenario Checking. The color segments of a bar represent the percentage of tasks that finally passed Eval2/Eval1/Eval0 or failed at Eval0. *w. pass@k* and *w/o. pass@k* represent the pass@k performance with or without the method, respectively.

combinational circuits. With the help of self-enhancement, especially the code standardization for sequential circuits, introduced in Section 3.2, the Eval0 pass@1 ratio of AutoBench reaches 97.33% (column 3, row 12), which is a huge improvement compared with 55.47% of the baseline (column 4, row 12). According to this comparison, AutoBench outperforms the baseline in all aspects.

5.2.3 Eval2 Coverage Distribution In certain instances of Eval1 pass rates (columns 11 and 12 in row 5), AutoBench performs slightly worse than the baseline ($88.46\% < 92.95\%$). This is due to the low testbench coverage of the baseline. An extreme example illustrates this: if a testbench reports a “pass” for any DUT, its Eval1 pass rate would be 100%. Therefore, Eval2 is required for a comprehensive assessment of testbenches. Eval2 is a coverage-based criterion and is deemed as *pass* when its ratio is over 80% in this work, as discussed in Section 4. To analyze the detailed coverage distribution of generated testbenches, the ones passing Eval1 from 1560 tasks (156×10 samples) were selected and the distribution of their Eval2 coverage is presented in Fig. 5. Compared with the baseline, Eval2 coverage of AutoBench is more concentrated on the right side, which means AutoBench is more capable of detecting errors in the DUTs than the baseline.

5.3 Ablation Study of Self-Enhancement

5.3.1 Impact of Auto-Debug To evaluate the impact of auto-debug as described in Section 3.2.3, the performance of AutoBench without debugging is compared with the original version with debugging of AutoBench as shown in Fig. 6(a). The color segments of a bar in Fig. 6(a) represent the percentage of tasks that finally passed Eval2/Eval1/Eval0 or failed at Eval0. Compared with the version without auto-debug, the full functional AutoBench has an about 8% improvement regarding pass@1 Eval2, and 21% more pass@1 Eval0 ratio.

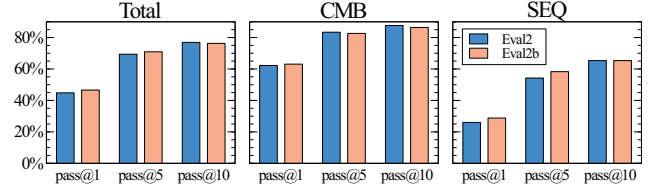


Figure 7: Performance on LLM-generated Verilog Codes.

5.3.2 Impact of Scenario Checking To assess the impact of scenario checking described in Section 3.2.2, a performance comparison between the presence and absence of scenario checking is shown in Fig. 6(b). Compared with the version without scenario checking, the fully functional AutoBench exhibits an approximate 10% improvement in pass@1 Eval2, along with 4.5% and 2.5% enhancements in pass@5 Eval2 and pass@10 Eval2.

5.4 Performance on LLM-Generated RTL Codes

The final evaluation metric in prior experiments, Eval2, utilizes the DUTs modified from the golden RTL solution using an LLM. With the increasing prevalence of LLM-assisted RTL code generation, potential challenges may arise when verifying RTL codes generated by the same LLM used in AutoBench. To test the feasibility of AutoBench in this context, we replaced the mutant DUTs in Eval2 with LLM-generated RTL codes using the same LLM. This revised metric, termed *Eval2b*, is assessed in Fig. 7. The labels at the top of the sub-figures represent the circuit type groups. Two bars in a group on each pass@k metric represent Eval2 (coverage metric on mutant codes from golden RTL) and Eval2b (coverage metric on LLM-generated codes), respectively. These results show that AutoBench performs similarly or even better on LLM-generated RTL codes compared with Eval2, indicating that our approach is viable for LLM-generated RTL codes.

5.5 Discussion

Although AutoBench is designed to generate testbench code without human intervention, it also demonstrates a greater potential for manual correction compared with the baseline. The testbench structure from AutoBench is more organized by test scenarios than the baseline, which usually consists of only simple stimuli and reference signals. The core checker codes are written in Python. Consequently, even a software engineer can rectify errors that arise in the checker component. Furthermore, the scenario-based driver can be easily extended by humans if any scenarios are still missing. In the future, more self-examination and self-correction approaches will be explored for AutoBench.

6 Conclusion

In this paper, we propose AutoBench and AutoEval, the LLM-automated hybrid testbench generation and evaluation frameworks for hardware simulation-based verification. AutoBench demonstrates a 57% improvement in the testbench Eval2 pass@1 ratio. For sequential circuits, AutoBench has 3.36 times Eval2 pass@1 ratio compared with the baseline.

Acknowledgments

This work is supported by the Deutsche Forschungsgemeinschaft (DFG, German Research Foundation) -Project-ID 504518248 and by TUM International Graduate School of Science and Engineering (IGSSE).

References

- [1] 2013. IEEE Standard for SystemVerilog–Unified Hardware Design, Specification, and Verification Language. *IEEE Std 1800-2012 (Revision of IEEE Std 1800-2009)* (2013).
- [2] Jason Blocklove, Siddharth Garg, Ramesh Karri, and Hammond Pearce. 2023. Chip-Chat: Challenges and Opportunities in Conversational Hardware Design. In *ACM/IEEE Workshop on Machine Learning for CAD (MLCAD)*.
- [3] Kaiyan Chang, Ying Wang, Haimeng Ren, Mengdi Wang, Shengwen Liang, Yinhe Han, Huawei Li, and Xiaowei Li. 2023. ChipGPT: How Far Are We From Natural Language Hardware Design. *arXiv preprint:2305.14019* (2023).
- [4] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. 2021. Evaluating Large Language Models Trained on Code. *arXiv preprint: 2107.03374* (2021).
- [5] Tinghuan Chen, Grace Li Zhang, Bei Yu, Bing Li, and Ulf Schlichtmann. 2023. Machine Learning in Advanced IC Design: A Methodological Survey. *IEEE Design & Test* 40 (2023), 17–33.
- [6] Hana Chockler, Orna Kupferman, and Moshe Y. Vardi. 2003. Coverage Metrics for Formal Verification. In *Correct Hardware Design and Verification Methods*. Berlin, Heidelberg, 111–125.
- [7] Harry Foster. 2022. The 2022 Wilson Research Group Functional Verification Study. <https://blogs.sw.siemens.com/verificationhorizons/2022/12/12/part-8-the-2022-wilson-research-group-functional-verification-study/>
- [8] Lei Huang, Weijiang Yu, Weitao Ma, Weihong Zhong, Zhangyin Feng, Haotian Wang, Qianglong Chen, Weihua Peng, Xiaocheng Feng, Bing Qin, and Ting Liu. 2023. A Survey on Hallucination in Large Language Models: Principles, Taxonomy, Challenges, and Open Questions. *arXiv preprint: 2311.05232* (2023).
- [9] Charalambos Ioannides and Kerstin I Eder. 2012. Coverage-Directed Test Generation Automated by Machine Learning – A Review. *ACM Transactions on Design Automation of Electronic Systems (TODAES)* 17 (2012), 1–21.
- [10] Nathan Kitchen and Andreas Kuehlmann. 2007. Stimulus Generation for Constrained Random Simulation. In *IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*.
- [11] Mingjie Liu, Nathaniel Pinckney, Bruce Khailany, and Haoxing Ren. 2023. VerilogEval: Evaluating large language models for verilog code generation. In *International Conference on Computer Aided Design (ICCAD)*.
- [12] Jack McEllin, Richard Conway, and Conor Ryan. 2022. AVERT: An Automatic Verilog Testbench Generation Tool for Grammatical Evolution. In *33rd Irish Signals and Systems Conference (ISSC)*.
- [13] Shahid Ali Murtza, Osman Hasan, and Kashif Saghar. 2016. VerT-Gen: An automatic verilog testbench generator for generic circuits. In *International Conference on Emerging Technologies (ICET)*.
- [14] OpenAI. 2024. OpenAI models API. <https://platform.openai.com/docs/models>
- [15] Marcelo Orenes-Vera, Margaret Martonosi, and David Wentzlauff. 2023. Using LLMs to Facilitate Formal Verification of RTL. *arXiv preprint: 2309.09437* (2023).
- [16] Khushboo Qayyum, Muhammad Hassan, Sallar Ahmadi-Pour, Chandan Kumar Jha, and Rolf Drechsler. 2024. LLM-assisted Automated Incremental Proof Generation for Hardware Verification. In *Design Automation Conference (DAC)*.
- [17] Ruixiang Tang, Dehan Kong, Longtao Huang, and Hui Xue. [n. d.]. Large Language Models Can be Lazy Learners: Analyze Shortcuts in In-Context Learning. In *Findings of the Association for Computational Linguistics: ACL 2023*.
- [18] Shailja Thakur, Jason Blocklove, Hammond Pearce, Benjamin Tan, Siddharth Garg, and Ramesh Karri. 2023. AutoChip: Automating HDL Generation Using LLM Feedback. *arXiv preprint: 2311.04887* (2023).
- [19] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. 2022. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems* 35 (2022), 24824–24837.
- [20] Stephen Williams. 2024. The ICARUS verilog compilation system. <https://github.com/steveicarus/iverilog>
- [21] Henry Wong. 2019. Problem sets - HDLBits. <https://hdlbits.01xz.net/wiki/Problemsets>
- [22] Chao Xiao, Yifei Deng, and Weixia Xu. 2024. LLM-Based Processor Verification: A Case Study for Neuromorphic Processor. In *Design, Automation and Test in Europe (DATE)*.
- [23] Shuo Yang, Robert Wille, and Rolf Drechsler. 2014. Determining Cases of Scenarios to Improve Coverage in Simulation-Based Verification. In *the 27th Symposium on Integrated Circuits and Systems Design*.
- [24] Shuo Yang, Robert Wille, and Rolf Drechsler. 2014. Improving Coverage of Simulation-Based Verification by Dedicated Stimuli Generation. In *Euromicro Conference on Digital System Design*.
- [25] Zixi Zhang, Greg Chadwick, Hugo McNally, Yiren Zhao, and Robert Mullins. 2023. LLM4DV: Using Large Language Models For Hardware Test Stimuli Generation. *arXiv preprint: 2310.04535* (2023).

Appendix: Artifact

6.1 Abstract

This is the artifact of paper *AutoBench: Automatic Testbench Generation and Evaluation Using LLMs for HDL Design*. All the code implementation, dataset and experimental results have been open-sourced. Please see <https://github.com/AutoBench/AutoBenchfor> for more detailed information about installation, setup and running.

6.2 Artifact check-list (meta-information)

- **Model:** OpenAI GPT-4-turbo, GPT-4O, GPT-4O-mini
- **Data set:** HDLBits problem set. Already in our project directory.
- **Run-time environment:** Linux, Python 3.8 or newer, the latest version of Icarus Verilog (IEEE Std 1800-2012 supported).
- **Hardware:** CPU with better performance than i7-11700k.
- **Metrics:** Already discussed in paper - syntactic and functional correctness (Eval0, Eval1 and Eval2)
- **Output:** code files, conversation files and log files.
- **Experiments:** please see the README file in <https://github.com/AutoBench/AutoBench>.
- **How much disk space required (approximately)?:** 4 GB
- **How much time is needed to prepare workflow (approximately)?:** 10 minutes.
- **How much time is needed to complete experiments (approximately)?:** one problem: 3 10 minutes; 156 problems: 13 hours.
- **Publicly available?:** Yes. All the codes, dataset and the experimental results are available to public.
- **Code licenses (if publicly available)?:** We sue GPL-3.0 license.
- **Workflow automation framework used?:** Yes, this is a totally automatic framework.

6.3 Description

6.3.1 How to access Please go to <https://github.com/AutoBench/AutoBench> and follow the detailed guidance. Here is the brief guidance:

- Firstly you need to install Python and Icarus Verilog. The version of Python should be at least 3.8. The version of Icarus Verilog should be the latest and perfectly support IEEE Std 1800-2012.
- Then please install the required Python libraries. Please see requirements.txt file in github project for the detailed information.

- Then please insert your LLM API key into config/key_API.json before running the project.
- Do not forget to change the bin path of iverilog you installed in iverilog_call.py.
- At last, you can run a CMB task demo via "python main.py -d cmb" to make sure you have correctly setup the project. Then you can run "python main.py -d 156" for the whole experiment, or run AutoBench on specific tasks according to the guidance in README.

6.3.2 *Hardware dependencies* Check README at <https://github.com/AutoBench/AutoBenchor> section 6.1 Hardware.

6.3.3 *Software dependencies* Check README at <https://github.com/AutoBench/AutoBenchor> subsection 6.3.1.

6.3.4 *Data sets* The data set we used is already included in our project directory. You can find it on <https://github.com/AutoBench/AutoBench/tree/master/data>.

6.3.5 *Models* We recommend you use the same LLM as mentioned in paper, i.e. the GPT-4-turbo. We do not guarantee other models will function flawlessly in our work.

6.4 Installation

You need to install Python and Icarus Verilog. The version of Python should be at least 3.8. The version of Icarus Verilog should be the latest and perfectly support IEEE Std 1800-2012. Then please install the required Python libraries. Please see requirements.txt file in github project for the detailed information.

6.5 Experiment workflow

Check README at <https://github.com/AutoBench/AutoBenchor> subsection 6.3.1.

6.6 Evaluation and expected results

The evaluation is automatically performed during running. See Section IV in our paper for the definitions of Eval0/Eval1/Eval2. The performance should be similar to the original experimental results in <https://drive.google.com/drive/folders/1EhG9Ch4vDzMtOsDvoiHthU0OWsZP1xRh?usp=sharing>. The Eval 2 pass@1 ratio may fluctuate by up to 5%.

6.7 Experiment customization

We provide multiple running choices. You can run AutoBench with pre-written configurations or customize your own running. Please see README on <https://github.com/AutoBench/AutoBench>.

Appendix: Code Examples for TB Generation

```
[problem description]:
Build a combinational circuit with 100 inputs, in[99:0]. There are 3 outputs:
// (1) out_and: output of a 100-input AND gate.
// (2) out_or: output of a 100-input OR gate.
// (3) out_xor: output of a 100-input XOR gate.

[module header]:
module top_module (input [99:0] in, output out_and, output out_or, output out_xor );

[Circuit Type] (Updated after Stage 0): CMB
```

Figure 8: A demo of Benchmark Suite.

```
Prompt:
1. Your task is to write a verilog testbench for a verilog RTL module code ... (omitted due to space constraints)

2. You are in the first stage. In this stage, please summarize the technical details of the DUT and give me a technical specification of the testbench generation task, so we can use it to design its corresponding testbench.

3. ... (omitted due to space constraints)

4. Your technical specification should include: ... (omitted due to space constraints)

5. Below is the information including the problem description and the DUT header:
[Benchmark Suite]

Your response must be in JSON form. example: { "important data": "...", # type: string. If no table, state transition or other direct data, leave this with "" "technical specifications": ["...", "...", "..."] # each element of the list is one specification string, the starting of the string is its index. }

LLM's Output:
[testbench specification]:

{"important data": "",
"technical specifications": [ "1. DUT Module Description: The Device Under Test (DUT) is defined in the Verilog module named 'top_module'. It contains a single input bus of width 100 bits (in[99:0]) and three separate output signals: out_and, out_or, and out_xor. These outputs represent the computed AND, OR, and XOR operations of all 100 inputs respectively.",
"2. Input Specification: The DUT accepts a 100-bit input vector. Each bit of this vector can independently be either 0 or 1. Comprehensive testing will involve applying various combinations of zeros and ones to these inputs.",
"3. ...", ... ]} (omitted due to space constraints)
```

Figure 9: A demo of Stage 1 in AutoBench. (Further demos are in the following pages.)

Prompt:

1. Your task is to write a verilog testbench for a verilog RTL module code ... (omitted due to space constraints)
2. You are in stage 2. In this section, please give me the test scenarios. you only need to describe the stimulus in each test scenario. If time is important, please provide the clock cycle information... (omitted due to space constraints)
3. Your information is:
[Benchmark Suite]
[testbench specification from stage 1]

You only need to describe the stimulus in each test scenario. If time is important, please provide the clock cycle information. we will use the stimulus description to generate the test vectors and send them to DUT. you must not tell the expected results even though you know that.

Your response must be in JSON form. example:

```
{ "scenario 1": "...", # each content is a string
  "scenario 2": "...", ... }
```

LLM's Output:

[scenario list]:

```
{ "scenario 1": "All input bits are set to 0 (000...000). This tests the basic operation when all inputs are at their lowest value.",
  "scenario 2": "All input bits are set to 1 (111...111). This scenario checks the outputs when all inputs are at their maximum.",
  ... (omitted due to space constraints) }
```

(a) stage 2

Prompt:

1. Your task is to write a verilog testbench for a verilog RTL module code ...
2. You are in stage 3; in this stage, please give me the core rules of an ideal DUT. you should give these rules in Python. (For convenience, you can use binary or hexadecimal format in Python, i.e., 0b0010 and 0x1a). Later we will use these ideal rules to generate expected values in each test scenario... (omitted due to space constraints)

3. Your information is:

[Benchmark Suite]

[testbench specification from Stage 1]

[test scenarios from stage 2]

Your response should only contain Python code. For convenience, you can use binary or hexadecimal format in Python. For example: 0b0010 and 0x1a

LLM's Output:

Here's the Python code, which will provide the expected results based on the input vector provided for each scenario.

```
import numpy as np
# code comments are omitted
def calculate_out_and(input_vector):
    return np.all(input_vector == 1)
def calculate_out_or(input_vector):
    return np.any(input_vector == 1)
def calculate_out_xor(input_vector):
    return np.sum(input_vector % 2 != 0)
```

(a) stage 3

Prompt:

1. Your task is to write a verilog testbench for a verilog RTL module code ... (omitted due to space constraints)
2. You are in stage 4... (omitted due to space constraints)
3. Your information is:
[Benchmark Suite]
[testbench specification from Stage 1]
[scenario list from stage 2]
The testbench does not need to check the DUT's output but only export them.

Instruction of saving signals to file:

- (1) You should use \$fopen and \$fdisplay to export the important signals in testbench. the file name is "TBout.txt".
- (2) When running testbench, for one time point, you should export 1 line. the example of the printed line is "scenario: 1, in = %d, out_and = %d, out_or = %d, out_xor = %d"... (omitted due to space constraints)
- (3) Attention: before \$fdisplay, you should always have a delay statement to make sure the signals are stable.
... (omitted due to space constraints)

LLM's Output:

```
... // omitted
// Instantiate the Device Under Test (DUT)
top_module DUT (.in(in), .out_and(out_and), .out_or(out_or),
               .out_xor(out_xor));
initial begin
    file_descriptor = $fopen("TBout.txt", "w");
    // Scenario 1: All bits are zero
    in = 100'b0;
    #10 $fdisplay(file_descriptor, "scenario:1,in=%d, out_and=%d, out_or=%d, out_xor=%d", in,
                  out_and, out_or, out_xor);
    // Scenario 2: All bits are one
    in = ~100'b0;
    #10 $fdisplay(file_descriptor, "scenario:2,in=%d, out_and=%d, out_or=%d, out_xor=%d", in,
                  out_and, out_or, out_xor);
    // Scenario 3: Alternating bits starting with 0
    in = 100'b0101010101_0101010101_0101010101_0101010101_0101010101_0101010101_0101010101_0101010101;
    #10 $fdisplay(file_descriptor, "scenario:3,in=%d, out_and=%d, out_or=%d, out_xor=%d", in,
                  out_and, out_or, out_xor);
    ... // omitted
```

(b) stage 4

Prompt:

1. Background: Your task is to verify the functional correctness of a verilog RTL module code... (omitted due to space constraints)
2. You are in the last stage... (omitted due to space constraints)

3. Your python script should contain a function "check_dut", its header is "def check_dut(test_vectors:list) → bool:". It can also call other functions you write in this script. If all test scenarios passed, function "check_dut" should return an empty list []. Otherwise, it should return the list of failed scenario indexes. You can use binary (like 0x1101), hexadecimal (like 0x1a), or normal number format in Python.

4. You have the information below to help you check the correctness of DUT:

[Benchmark Suite]

[testbench specification from stage 1]

[checking rules in python from stage 3]

important instructions replay: ... (omitted due to space constraints)

please only generate the Python codes, no other words.

LLM's Output:

```
import numpy as np
def calculate_out_and(input_vector):
    return np.all(input_vector == 1)
def calculate_out_or(input_vector):
    return np.any(input_vector == 1)
def calculate_out_xor(input_vector):
    return np.sum(input_vector % 2 == 1)
def check_dut(test_vectors):
    failed_scenarios = []
    for idx, scenario in enumerate(test_vectors):
        input_value = scenario['in']
        input_vector = np.array([int(x) for x in np.binary_repr(input_value, width=100)])
        expected_and = calculate_out_and(input_vector)
        expected_or = calculate_out_or(input_vector)
        expected_xor = calculate_out_xor(input_vector)
        actual_and = bool(scenario['out_and'])
        actual_or = bool(scenario['out_or'])
        actual_xor = bool(scenario['out_xor'])
        if (expected_and != actual_and or expected_or != actual_or or expected_xor != actual_xor):
            failed_scenarios.append(idx)
    return failed_scenarios
```

(b) stage 5

Figure 10: Demos of the Driver Track: (a) Stage 2; (b) Stage 4

Figure 11: Demos of Python Checker Track: (a) Stage 3; (b) Stage 5

Prompt:

You already have the code generated from stage 4 part 1:
[codes from Fig. 12]

Please help me to export the input of DUT module by using code below:

[IMPORTANT]: \$fdisplay(file, "scenario: %d, clk = %d, load = %d, ena = %d, amount = %d, data = %d, q = %d", scenario, clk, load, ena, amount, data, q);

You should insert the code above into scenario checking part. In each scenario, you should insert the code above after the input of DUT module changed. Don't delete the existing \$fdisplay codes.

For example, for a circuit that has two input signals changed at different times in one scenario, the original code is like this: - original code: // scenario 1 begins

```
scenario = 1;
signal_1 = 1;
// insert $fdisplay here
#10;
```

```
signal_2 = 1;
// insert $fdisplay here
#10;
```

```
$fdisplay(file, "[check]scenario: %d, clk = %d, load = %d, ena = %d, amount = %d, data = %d, q = %d", scenario, clk, load, ena, amount, data, q);
#10;
```

```
// scenario 1 ends
...(omitted due to space constraints)
```

Please insert codes according to the rules above. DO NOT modify other codes! Please reply with the modified full codes. please only reply with Verilog codes, no other words.

LLM's Output:

```
... // omitted
// Scenario Based Test
initial begin
    file = $fopen("TBout.txt", "w")
    // scenario 1
    scenario = 1; load = 1; ena = 0;
    data = 64'hFFFFFFFFFFFFFFFF;
    $fdisplay(file, "scenario:_%d,_clk=_%d,_load=_%d,_
        ena=_%d,_amount=_%d,_data=_%d,_q=_%d",
        scenario, clk, load, ena, amount, data, q);
    #10;
    $fdisplay(file, "[check]scenario:_%d,_clk=_%d,_load=_%d,
        _ena=_%d,_amount=_%d,_data=_%d,_q=_%d",
        scenario, clk, load, ena, amount, data, q);
    #10;
    // scenario 2
    scenario = 2; load = 0; ena = 1;
    amount = 2'b00;
    $fdisplay(file, "scenario:_%d,_clk=_%d,_load=_%d,_
        ena=_%d,_amount=_%d,_data=_%d,_q=_%d",
        scenario, clk, load, ena, amount, data, q);
    #10;
    $fdisplay(file, "[check]scenario:_%d,_clk=_%d,_load=_%d,
        _ena=_%d,_amount=_%d,_data=_%d,_q=_%d",
        scenario, clk, load, ena, amount, data, q);
    ... // omitted
```

Figure 13: A demo of Stage 4 part 2 in AutoBench for sequential DUT.

Prompt:

1. Your task is to complete a given verilog testbench code. This testbench is for a verilog RTL module code (we call it as "DUT", device under test). This circuit is a **sequential circuit**. ...(omitted due to space constraints)

Your information is:

[problem description from Benchmark Suite]

[module header from Benchmark Suite]

[testbench specification from Stage 1]

[scenario list from stage 2]

The testbench does not need to check the DUT's output but only exports the signals of the DUT. Please export the signals of DUT to a file named "TBout.txt" at the end of each scenario. The template is given below:

```
$fdisplay(file, "[check]scenario: %d, clk = %d, load = %d, ena = %d, amount = %d, data = %d, q = %d", scenario, clk, load, ena, amount, data, q);
```

The variables are already declared. The clock signal is already prepared. This output will be used to check the correctness of the DUT's output later.

Please only use "#10" as the delay when needed. If you need a longer delay, you can use multiple "#10", such as "#10; #10; #10;". Avoid meaningless long delays in your code.

...(omitted due to space constraints)

Please determine the input signal's exact values according to the test scenarios.

Note: please complete the last initial code part (marked in the given testbench template). You should give me the completed full code. The testbench template above is to help you generate the code. You must use %d when exporting values.

Please generate the full testbench code. please only reply Verilog codes, no other words.

LLM's Output:

```
... // omitted
// DUT instantiation
top_module DUT (.clk(clk), .load(load), .ena(ena), .amount(
    amount), .data(data), .q(q));
// Clock generation
initial begin
    clk = 0;
    forever #5 clk = ~clk;
end
// Scenario Based Test
initial begin
    file = $fopen("TBout.txt", "w")
    // scenario 1
    scenario = 1; load = 1; ena = 0;
    data = 64'hFFFFFFFFFFFFFFFF;
    #10;
    $fdisplay(file, "[check]scenario:_%d,_clk=_%d,_load=_%d,
        _ena=_%d,_amount=_%d,_data=_%d,_q=_%d",
        scenario, clk, load, ena, amount, data, q);
    #10;
    // scenario 2
    scenario = 2; load = 0; ena = 1;
    amount = 2'b00;
    #10;
    $fdisplay(file, "[check]scenario:_%d,_clk=_%d,_load=_%d,
        _ena=_%d,_amount=_%d,_data=_%d,_q=_%d",
        scenario, clk, load, ena, amount, data, q);
    ... // omitted
```

Figure 12: A demo of Stage 4 part 1 in AutoBench for sequential DUT, taking sequential task *shift18* as an example.